



Structuri de Date și Algoritmi (SDA)

As.Drd.Ing. Bianca Gușiță

AC/TI, AN II

LAB 12-13 – Recursivitate





Recursivitatea este o tehnică de programare în care o funcție se apelează pe ea însăși pentru a rezolva o problemă.

- În C, recursivitatea este utilizată pentru a descompune problemele complexe în subprobleme mai mici și mai ușor de rezolvat. Recursivitatea este folosită în special pentru problemele care pot fi împărțite în subprobleme similare cu problema inițială (ex. factorizarea unui număr, calculul unui termen din seria Fibonacci etc.).

Elementele unei funcții recursive

- O funcție recursivă are două părți esențiale:
 - 1.Cazul de bază (condiția de oprire)** – este o condiție în care funcția nu se mai apelează pe ea însăși, oprind astfel recursivitatea. Fără un caz de bază, funcția recursivă ar intra într-o buclă infinită.
 - 2.Cazul recursiv** – este partea în care funcția se apelează pe ea însăși, dar cu un argument redus sau simplificat, astfel încât fiecare apel recursiv să se apropie de cazul de bază.



Laborator 12 – Recursivitate

Problema turnurilor din Hanoi este o problemă clasică de recursivitate.

- Avem n discuri și trei tije (A, B și C). Scopul este să mutăm toate discurile de pe tija A pe tija C, respectând următoarele reguli:
- Putem muta doar un disc o dată.
- Nu putem pune un disc mai mare peste un disc mai mic.

Explicație

- Cazul de bază este atunci când avem un singur disc, care se mută direct de la tija from la tija to.
- În cazul recursiv, mutăm $n-1$ discuri de la from la aux, mutăm discul n de la from la to, apoi mutăm cele $n-1$ discuri de la aux la to.

```
#include <stdio.h>
```

```
// Funcția recursivă pentru turnurile din Hanoi
void hanoi(int n, char from, char to, char aux) {
    if (n == 1) { // Cazul de bază: un singur disc
        printf("Mută discul 1 de la %c la %c\n", from, to);
        return;
    }
    // Mutăm n-1 discuri de la 'from' la 'aux' folosind 'to' ca ajutor
    hanoi(n - 1, from, aux, to);
    printf("Mută discul %d de la %c la %c\n", n, from, to);
    // Mutăm n-1 discuri de la 'aux' la 'to' folosind 'from' ca ajutor
    hanoi(n - 1, aux, to, from);
}
```

```
int main() {
    int num;
    printf("Introduceți numărul de discuri: ");
    scanf("%d", &num);
    printf("Soluția pentru turnurile din Hanoi cu %d discuri:\n", num);
    hanoi(num, 'A', 'C', 'B');
    return 0;
}
```



Backtracking este o tehnică de căutare și explorare folosită în programare pentru a găsi toate soluțiile posibile la o problemă, prin încercarea sistematică a fiecărei opțiuni și "retragerea" în cazul în care opțiunea curentă nu conduce la o soluție.

- În C, backtracking este adesea implementat folosind recursivitate și este util pentru problemele de tipul "combinări" sau "alegeri", cum ar fi Sudoku, problema reginelor sau generarea tuturor subseturilor unui set.

Elementele unei soluții de backtracking

- 1.Cazul de bază:** Situația în care o soluție a fost găsită și trebuie gestionată corespunzător (de exemplu, prin afișare sau stocare).
- 2.Cazul recursiv:** Funcția încearcă fiecare opțiune disponibilă, și dacă opțiunea respectivă este validă, face un apel recursiv. Dacă opțiunea nu duce la o soluție, funcția se întoarce și încearcă următoarea opțiune.

Laborator 12 – Recursivitate



Problema reginilor (**N-Queens Problem**) este o problemă clasică de backtracking.

- Scopul este să plasăm N regine pe o tablă de șah de $N \times N$ astfel încât nicio două regine să nu se atace reciproc.
- Două regine se atacă reciproc dacă se află pe aceeași linie, coloană sau diagonală.

Explicația codului

- 1.Funcția este_valid:** Verifică dacă o regină poate fi plasată pe o anumită poziție (row, col). Este valid să plasăm o regină dacă nu există o altă regină pe aceeași coloană, diagonala stânga-sus sau diagonala dreapta-sus.
- 2.Funcția plaseaza_regine:** Este funcția principală de backtracking. Încearcă să plaseze reginele pe rând, pe fiecare linie (row). Dacă o poziție este validă, plasează regina și apelează recursiv funcția pentru următoarea linie. Dacă nu se găsește o soluție în continuare, elimină regina plasată anterior (backtracking) și continuă cu următoarea coloană.
- 3.Funcția afisare_tabla:** Afișează tabla de șah cu reginele plasate.
- 4.Funcția main:** Inițializează tabla, apelează funcția de backtracking și afișează soluția, dacă există una.

Laborator 12 – Recursivitate - Backtracking



```
#include <stdio.h>
#include <stdlib.h>

#define N 8 // Dimensiunea tablei de șah

// Funcția pentru a verifica dacă o regină poate fi plasată pe tabla de șah
int este_valid(int tabla[N][N], int row, int col) {
    int i, j;

    // Verifică coloana
    for (i = 0; i < row; i++)
        if (tabla[i][col])
            return 0;

    // Verifică diagonală superioară stânga
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (tabla[i][j])
            return 0;

    // Verifică diagonală superioară dreapta
    for (i = row, j = col; i >= 0 && j < N; i--, j++)
        if (tabla[i][j])
            return 0;

    return 1;
}
```

Laborator 12 - Recursivitate - Backtracking



```
// Funcția recursivă de backtracking pentru plasarea reginelor
int plaseaza_regine(int tabla[N][N], int row) {
    if (row >= N) {
        // Cazul de bază: toate reginele sunt plasate
        return 1;
    }

    for (int col = 0; col < N; col++) {
        // Verificăm dacă regina poate fi plasată la (row, col)
        if (este_valid(tabla, row, col)) {
            tabla[row][col] = 1; // Plasăm regina

            // Apel recursiv pentru următoarea linie
            if (plaseaza_regine(tabla, row + 1))
                return 1;

            // Dacă plasarea reginei nu duce la o soluție, o eliminăm (backtrack)
            tabla[row][col] = 0;
        }
    }

    return 0;
}
```

Laborator 12 - Recursivitate - Backtracking



```
// Funcția pentru a afișa tabla de șah
void afisare_tabla(int tabla[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%s ", tabla[i][j] ? "R" : ".");
        }
        printf("\n");
    }
}

int main() {
    int tabla[N][N] = {0}; // Inițializăm tabla cu zero

    if (plaseaza_regine(tabla, 0)) {
        printf("O soluție pentru problema reginelor:\n");
        afisare_tabla(tabla);
    } else {
        printf("Nu există soluție pentru problema reginelor.\n");
    }

    return 0;
}
```

Laborator 12 – Recursivitate - Backtracking



Avantaje:

- 1.Elegant și intuitiv:** Permite rezolvarea problemelor complexe prin divizarea acestora în subprobleme mai mici și încercarea sistematică a fiecărei opțiuni.
- 2.Afișează toate soluțiile:** În problemele unde există multiple soluții (de exemplu, toate aranjamentele posibile), backtracking-ul poate fi utilizat pentru a le găsi pe toate.

Dezavantaje:

- 1.Ineficiență:** Deoarece backtracking-ul testează fiecare opțiune posibilă, poate fi ineficient pentru problemele de mari dimensiuni.
- 2.Consum mare de resurse:** Fiecare apel recursiv ocupă memorie în stivă, iar pentru probleme mari poate apărea o eroare de depășire a stivei (stack overflow).
- 3.Necesită optimizare:** În probleme complexe, este necesară utilizarea unor optimizări suplimentare (de exemplu, memoizarea) sau a unor euristici pentru a reduce spațiul de căutare.



Vă mulțumesc pentru
prezență și atenție!

As.drd.ing. Bianca Gușiță